Stages 1 and 2: the frozen models, 4B against 8B

2026-11-01

Stages 1 and 2 put the brief in front of a frozen language model, with no training, and score the controller it writes exactly as training will score it. Stage 1 gives the model one turn: the brief without the probe section, and the controller straight away. Stage 2 gives it the two-turn protocol: an optional probe run on a throwaway batch, whose printed output comes back to it, then the controller. This document compares two models on the same batches, `Qwen3-4B-Instruct-2507`, the base the training stages use, and `Qwen3-8B` in non-thinking mode, twice its size from the same generation. Both are read against the references Stage 0 fixed on the same held-out tasks.

Terms. A *sample* is one rollout on one task at the sampling temperature. *Valid* means the reply held a controller that passed validation (a `def policy(state)` with only the allowed imports). *Success* and *fast success* are the environment's own outcome and success within 1.25× the oracle's steps; *normalised time* is oracle steps over own steps, zero on failure. The *ride ratio* is the highest reported σ of the final run divided by the batch's limit: one is the oracle, lower is caution, above one is a shower. The *probe rate* is the share of Stage 2 samples whose first reply was probe code rather than a controller. The *estimate error* is how far the number closest to the true limit in the probe's printed output lies from it. *Cut off* means the reply opened a code block the token budget closed before it ended. *Ruined* is a shower or lost seeds; *error* is a controller that did not run, scored at the error floor.

1 Protocol

- **Tasks.** The 48 held-out tasks of `crystallizer_eval.jsonl`, 16 per third of the limit's range, the same batches as Stage 0's references.
- **Samples.** 4 per task per run at temperature 0.8, top-p 0.95, at most 1000 new tokens per turn.
- **Brief.** The brief of the paper's appendix, with the two quench logs and the stated range of the limit; Stage 1 without the probe section.
- **Models.** `Qwen3-4B-Instruct-2507` (instruct-only, no thinking mode) and `Qwen3-8B` with thinking switched off through the chat template's `enable_thinking=False`, as its model card instructs; bf16 weights, one A30 each, no gradient updates.
- **Scoring.** The runner's sandbox: the probe run on the probe batch with the limit masked, the controller on the scoring batch, `score_sample` as the trainer uses it.
- **References.** The cautious blind controller and the oracle of Stage 0, on the same tasks.

2 Results

Tables 1 to 4 carry every number; the reading follows them. The commit, job and result files behind each table are in the comments of this file and in the header of each table fragment.

Run	Model	Valid	Success	Fast	Norm. time	Reward	Ride ratio	Probe rate	Est. error
stage1-4b	iter_000	91.1%	27.1%	1.0%	0.156	−84.3	0.75	–	–
stage2-4b	iter_000	98.4%	23.4%	0.0%	0.119	−65.7	0.79	6.2%	–
stage1-8b	qwen3-8b	94.8%	5.7%	0.5%	0.032	−207.4	0.30	–	–
stage2-8b	qwen3-8b	98.4%	8.9%	1.0%	0.047	−191.8	0.50	5.7%	–
cautious blind reference	–	–	–	4.2%	0.493	38.4	–	–	–
oracle	–	–	–	100.0%	1.000	79.7	–	–	–

Table 1: The four runs on the held-out tasks, next to the Stage 0 references. Ride ratio, probe rate and estimate error are means over the samples that have them.

Run	n	Reply chars	Cut off	Success	Ruined	Timeout	Error	Probe chars	Probe crashed	Probe cut off	Est. within 0.01
stage1-4b	192	2999	17.7%	52	93	25	22	–	–	–	–
stage2-4b	192	2086	3.6%	45	132	11	4	2563–4189	0.0%	100.0%	0.0%
stage1-8b	192	2581	9.4%	11	39	51	91	–	–	–	–
stage2-8b	192	2504	4.7%	17	54	31	90	2813–4430	0.0%	100.0%	0.0%

Table 2: Where the samples end. The median length of the final reply in characters, the share of replies cut off by the token budget, counts of samples by outcome; for Stage 2 the length range of the probe replies and the share of probes that crashed, that were cut off by the token budget, and whose output held a number within 0.01 of the true limit.

2.1 Reading

1. **Both frozen models sit far below the cautious blind reference.** The 4B base scores −84 in one turn and −66 with the two-turn protocol; the 8B scores −207 and −192. The cautious blind controller, which never probes and never fails, scores +38; the oracle +80; the hand-written probe run and controller of Stage 0, +77. No run reaches fast success on more than one sample in a hundred. The headroom for training is the whole scale.

2. **The 8B is worse than the 4B on every column, and the reason is in Table 3:** 91 and 90 of its 192 samples never run. Its controllers use `L` without reading it from the state or refer to an `initial_L` they never set (49 undefined names in Stage 1); they read `state['done']` although the brief states that `done` is returned by `env.step` and the flags are attributes of `env` (7 crashes in Stage 1, 33 in Stage 2); and in Stage 2 they call `env` inside the controller although the brief states the controller has no access to it (37 controllers contain `env.step` or `env.reset`, 21 crash on it). In non-thinking mode the 8B writes longer, more elaborate code with more invented names. The 4B's errors are almost all replies the token budget cut off mid-code (16 of 22 in Stage 1).

3. **Neither model uses the probe run.** 12 and 11 first replies of 192 are probe code, and every one of them was cut off by the 1000-token budget: the sandbox rejected the unfinished block, the model received the rejection as its probe output, and wrote the controller without a measurement. No probe output holds a number within 0.01 of the limit, so the estimate-error column is empty and the Stage 2 ride ratios carry nothing a probe found. The probe replies run from 2 563 to 4 430 characters (Table 2), past the budget; the frozen probe rate is measured with the budget as a confound.

4. **For the 4B, the two-turn protocol helps at inference only by shortening the reply.** Reward rises from −84 to −66 because cut-off replies fall from 17.7% to 3.6% (median reply 2 999 characters in Stage 1, 2 086 in Stage 2), while success falls from 27% to 23% and fast success to zero.

5. **Where the samples succeed** (Table 4): the 4B succeeds on about half the high-third batches and on one in twenty to thirty of the low third. It cools at a fixed moderate rate that high limits tolerate and low limits punish: 93 and 132 batches ruined, and a mean ride ratio of 0.75 to 0.79 that averages over-riding low limits with under-riding high ones. The 8B

	stage1-4b	stage2-4b	stage1-8b	stage2-8b
samples that did not run	22	4	91	90
syntax error (reply cut off or malformed)	16	3	9	1
forbidden name at validation	1	0	1	2
reads state['done'], ['showered'] or ['failure_reason']	0	0	7	33
calls env in the controller	0	1	0	21
undefined name (NameError)	3	0	49	16
missing state key (KeyError)	0	0	17	5
local used before assignment	2	0	6	4
sandbox output parse error (the bug fixed on 2026-11-01)	0	0	0	0
other runtime error	0	0	2	8
controllers that call env.reset or env.step, of 192	0	6	0	37
controllers that read state['done'], ['showered'] or ['failure_reason'], of 192	0	0	7	35
controllers that print, of 192	0	5	11	38

Table 3: Why the samples that did not run failed, in fixed kinds assigned by the renderer from the error text, and three code patterns counted over all samples whether the controller ran or not. The kinds are exclusive and sum to the error count of Table 2.

		stage1-4b			stage2-4b			stage1-8b			stage2-8b		
Third	n	Fast	Succ.	Reward	Fast	Succ.	Reward	Fast	Succ.	Reward	Fast	Succ.	Reward
low	64	0.0%	3.1%	−124.1	0.0%	4.7%	−90.5	0.0%	4.7%	−209.5	0.0%	4.7%	−194.4
middle	64	1.6%	25.0%	−80.2	0.0%	20.3%	−66.6	0.0%	3.1%	−217.9	0.0%	6.2%	−187.8
high	64	1.6%	53.1%	−48.7	0.0%	45.3%	−39.8	1.6%	9.4%	−194.8	3.1%	15.6%	−193.3

Table 4: By third of the limit's range.

mostly times out (51 and 31 of 192): it cools too slowly or stops; its ride ratio is 0.30 to 0.50.

6. **The superseded first round agrees within sampling noise** (4B −85 and −53, 8B −209 and −195; the provenance comments in the source of this document name that round), so the sandbox bug floored twelve samples but did not shape the picture.

For the ladder. The 4B base remains the model to train: the 8B gives no better start at twice the memory. "Whether to probe" has a frozen baseline of about six percent, and at 1000 tokens per turn no probe the frozen models write fits; raising the budget, or measuring the probe rate at a budget where a probe can run, is a protocol decision to take before the training stages, not one this document takes.

3 Reproduce

The weights: `hf download Qwen/Qwen3-8B --local-dir models/hf_base/qwen3-8b` on the cluster's run node (the 4B base is at `models/hf_base/iter_000`). Each run is one job:

```
hpc submit lora4vr jobs/hpi/stage1.sbatch --job-name=stage1-8b \
    --export=ALL,LORA4VR_GIT_COMMIT=$(git rev-parse HEAD),MODEL_PATH=models/hf_base/qwen3-8b,N_SAMPLES=4
hpc submit lora4vr jobs/hpi/stage2.sbatch --job-name=stage2-8b \
    --export=ALL,LORA4VR_GIT_COMMIT=$(git rev-parse HEAD),MODEL_PATH=models/hf_base/qwen3-8b,N_SAMPLES=4
hpc pull lora4vr/results/stage2-8b_<jobid> docs/stage12/
uv run python scripts/frozen_render.py --out docs/stage12/tables \
    stage1-4b=docs/stage12/stage1-4b_<jobid> stage2-4b=docs/stage12/stage2-4b_<jobid> \
    stage1-8b=docs/stage12/stage1-8b_<jobid> stage2-8b=docs/stage12/stage2-8b_<jobid>
```

and the same with `MODEL_PATH=models/hf_base/iter_000` for the 4B runs. Each result directory holds the job log, `stage<N>_summary.json` with its provenance block and per-task rows, and `stage<N>_samples.jsonl` with every sample: the full transcript, the probe code, the tool output, the controller and its outcome.